

Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 2



Nama:

Muhamad Rizqi Candra

NIM:

20240801035

UNIVERSITASA ESA UNGGUL
TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER

2026

Pendahuluan

Pada praktikum pertemuan 2 ini saya mencoba membuat tampilan aplikasi Flutter yang lebih terstruktur. Materi yang saya praktikkan meliputi penggunaan beberapa widget dasar seperti Container, Row, Column, Expanded, ListView.builder, Card, dan ListTile. Selain itu, saya juga mencoba membuat data menggunakan class Dart dan mempelajari cara berpindah dari halaman daftar ke halaman detail menggunakan Navigator.

1. Tujuan Praktikum

- Memahami penggunaan Container, Padding, Row, Column, dan Expanded.
- Menampilkan data menggunakan ListView.builder.
- Membuat model data sederhana menggunakan class Dart.
- Mencoba navigasi antarhalaman menggunakan Navigator.push() dan Navigator.pop().

2. Teori Dasar

- Container : digunakan sebagai wadah untuk mengatur ukuran, warna, border, radius, padding, dan margin.
- Padding : digunakan untuk memberikan jarak antara isi widget dengan bagian di sekitarnya.
- Row / Column : Row digunakan untuk menyusun widget secara horizontal, sedangkan Column secara vertikal.
- Expanded : digunakan supaya widget dapat memakai sisa ruang yang tersedia di dalam Row atau Column.
- ListView.builder : digunakan untuk membuat daftar item secara dinamis, terutama jika jumlah datanya cukup banyak.
- Card + ListTile : digunakan untuk membuat tampilan item berbentuk kartu yang berisi ikon, judul, subjudul, dan ikon di bagian kanan.
- Navigator : digunakan untuk mengatur perpindahan halaman. Navigator.push() digunakan untuk membuka halaman baru, sedangkan Navigator.pop() untuk kembali.

3. Langkah Praktikum

Pertama saya membuka terminal, kemudian membuat project Flutter baru untuk digunakan pada praktikum ini.

```
flutter create praktikum2
```

```
cd praktikum2
```

```
flutter run
```

4. Implementasi Layout Kartu Profil

```

import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 2',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const ProfilePage(),
    );
  }
}

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Profil'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: Colors.blue.shade50,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Row(
            children: [
              const CircleAvatar(
                radius: 32,
                child: Icon(
                  Icons.person,
                  size: 32,
                ),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: const [
                    Text(
                      'Rizqi',
                      style: TextStyle(
                        fontSize: 20,
                        fontWeight: FontWeight.bold,
                      ),
                    ),

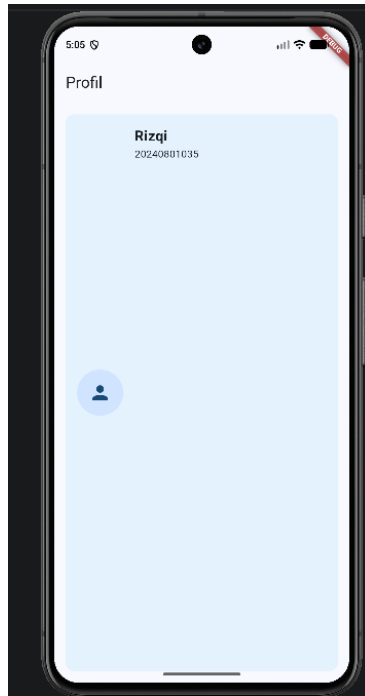
```

```

),
Text('20240801035'),
],
),
],
),
),
),
),
),
);
}
}

```

Pada bagian ini saya membuat halaman profil sederhana. Fungsi main() menjalankan MyApp, kemudian MyApp menampilkan ProfilePage. Di ProfilePage saya menggunakan Scaffold dan AppBar dengan judul "Profil". Bagian isi menggunakan Padding dan Container supaya tampilan kartu memiliki jarak dan sudut yang lebih rapi. Di dalam Container terdapat Row yang berisi CircleAvatar dan data nama serta NIM. Expanded digunakan supaya bagian teks dapat menyesuaikan ruang yang tersedia.



5. Implementasi Model Data dan ListView

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
}

```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'Praktikum 2',
    theme: ThemeData(
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    home: const MenuPage(),
  );
}

```

```

class Makanan {
  final String nama;
  final int harga;

  const Makanan(this.nama, this.harga);
}

```

```

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];

```

```

class MenuPage extends StatelessWidget {
  const MenuPage({super.key});
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Daftar Menu'),
    ),
    body: ListView.builder(
      itemCount: daftarMenu.length,
      itemBuilder: (context, index) {
        final item = daftarMenu[index];

        return Card(
          margin: const EdgeInsets.symmetric(
            horizontal: 12,

```

```

        vertical: 6,
      ),
      child: ListTile(
        leading: const Icon(Icons.restaurant),
        title: Text(item.nama),
        subtitle: Text('Rp ${item.harga}'),
        trailing: const Icon(Icons.chevron_right),
      ),
    );
  },
),
);
}
}

```

Pada bagian ini saya membuat data menu menggunakan class Makanan. Setiap objek memiliki nama dan harga. Data tersebut dimasukkan ke dalam daftarMenu, kemudian ditampilkan menggunakan ListView.builder. Setiap item dibuat menggunakan Card dan ListTile, sehingga daftar menu terlihat lebih sederhana dan bisa di-scroll. Karena data yang digunakan masih tetap, halaman ini masih menggunakan StatelessWidget.

6. Implementasi Navigasi ke Halaman Detail

```

import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class Makanan {
  final String nama;
  final int harga;

  const Makanan(this.nama, this.harga);
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(

```

```

        title: 'Praktikum 2',
        theme: ThemeData(
          colorSchemeSeed: Colors.blue,
          useMaterial3: true,
        ),
        home: const MenuPage(),
      );
    }
  }
}

class MenuPage extends StatelessWidget {
  const MenuPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Menu'),
      ),
      body: ListView.builder(
        itemCount: daftarMenu.length,
        itemBuilder: (context, index) {
          final item = daftarMenu[index];

          return Card(
            margin: const EdgeInsets.symmetric(
              horizontal: 12,
              vertical: 6,
            ),
            child: ListTile(
              leading: const Icon(Icons.restaurant),
              title: Text(item.nama),
              subtitle: Text('Rp ${item.harga}'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => DetailPage(
                      makanan: item,
                    ),
                  ),
                );
              },
            ),
          );
        },
      ),
    );
  }
}

```

```

    },
  ),
);
}
}

class DetailPage extends StatelessWidget {
  final Makanan makanan;

  const DetailPage({
    super.key,
    required this.makanan,
  });

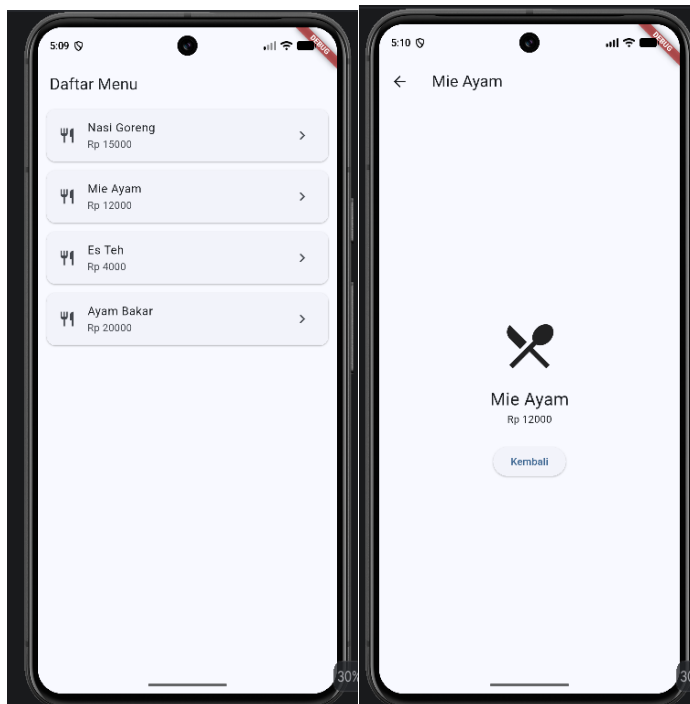
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(makanan.nama),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.restaurant_menu,
              size: 80,
            ),
            const SizedBox(height: 16),
            Text(
              makanan.nama,
              style: const TextStyle(
                fontSize: 24,
              ),
            ),
            const SizedBox(height: 24),
            Text('Rp ${makanan.harga}'),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali'),
            ),
          ],
        ),
      ),
    );
  }
}

```

```
}
}
```

Pada tahap ini saya menambahkan fungsi agar setiap menu bisa ditekan dan membuka halaman detail. Saat item dipilih, `Navigator.push()` digunakan untuk membuka `DetailPage` dan objek `Makanan` yang dipilih dikirim melalui parameter. Di halaman detail saya menampilkan nama makanan, harga, ikon, dan tombol Kembali. Tombol tersebut menggunakan `Navigator.pop()` untuk kembali ke halaman daftar.

Hasil akhir:



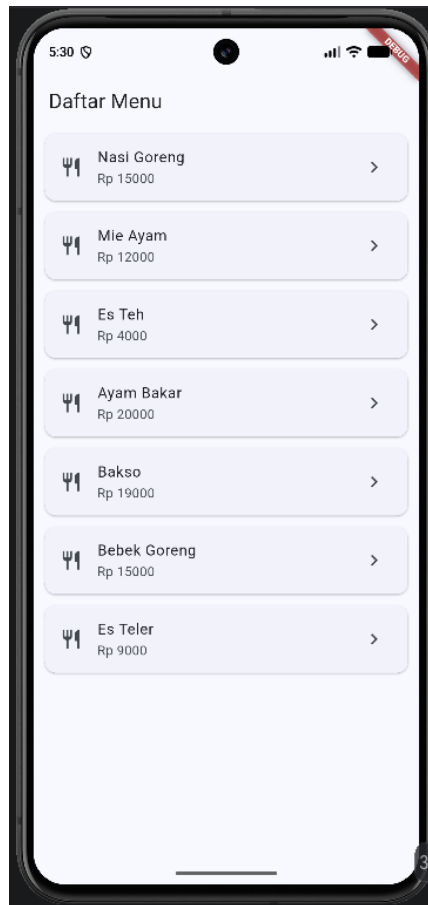
7. Latihan Mandiri

- Menambahkan 3 menu baru ke `daftarMenu` dan memastikan daftar masih bisa di-scroll.

```
const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
  Makanan('Bakso', 19000),
  Makanan('Bebek Goreng', 15000),
  Makanan('Es Teler', 9000),
];
```

Pada latihan ini saya menambahkan tiga menu baru sehingga jumlah data yang sebelumnya 4 menjadi 7. Bagian kode yang lain tidak saya ubah karena

ListView.builder sudah menyesuaikan jumlah item secara otomatis. Setelah dijalankan, semua menu dapat ditampilkan dan daftar tetap bisa di-scroll.



- Menambahkan properti deskripsi pada class Makanan dan menampilkannya di DetailPage.

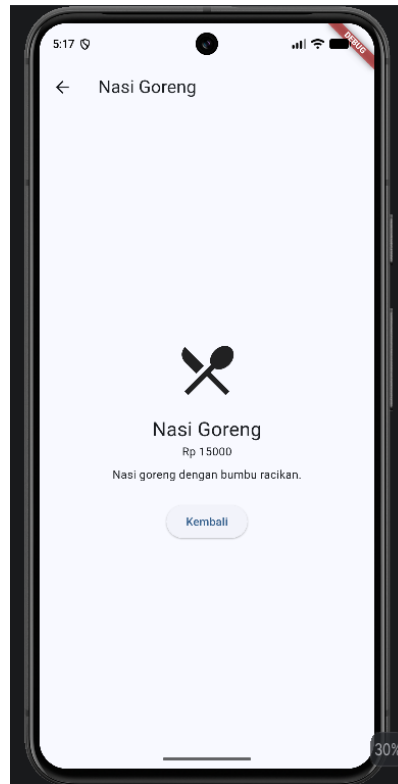
Pada latihan ini saya menambahkan satu properti baru yaitu deskripsi pada class Makanan. Jadi data makanan sekarang tidak hanya menyimpan nama dan harga, tetapi juga deskripsi. Setelah data dikirim ke DetailPage, deskripsi tersebut ikut ditampilkan di halaman detail.

```
class Makanan {  
    final String nama;  
    final int harga;  
    final String deskripsi;
```

```
    const Makanan(  
        this.nama,  
        this.harga,  
        this.deskripsi,  
    );  
}
```

```
const daftarMenu = [
  Makanan(
    'Nasi Goreng',
    15000,
    'Nasi goreng dengan bumbu racikan.',
  ),
]
```

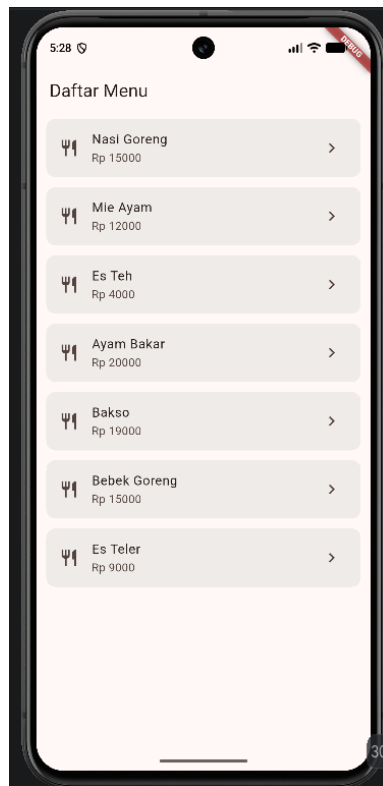
Hasil akhirnya:



- Mengganti Card dengan Container yang memiliki warna latar dan sudut membulat.

```
return Container(
  margin: const EdgeInsets.symmetric(
    horizontal: 12,
    vertical: 6,
  ),
  decoration: BoxDecoration(
    color: Colors.brown.shade50,
    borderRadius: BorderRadius.circular(12)
  ),
)
```

Di bagian ini saya mengganti Card dengan Container. Saya memberikan warna latar coklat shade 50 dan menggunakan `borderRadius.circular()` supaya sudut Container terlihat membulat.

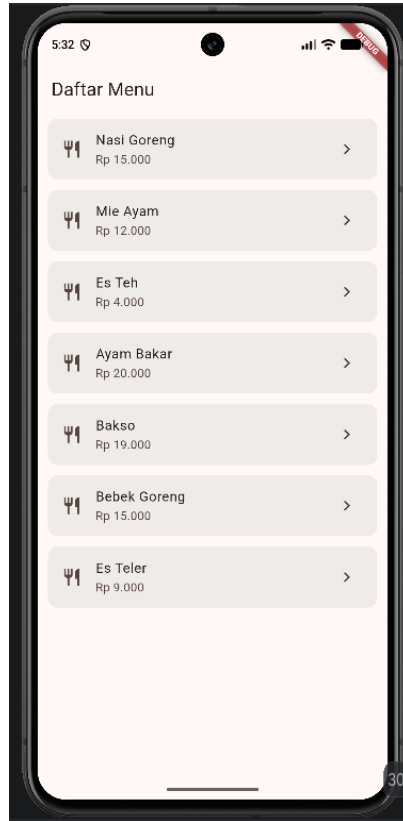


- Menampilkan harga dengan format ribuan, misalnya 15.000.

```
String formatRibuan(int angka) {  
    String a = angka.toString();  
    String hasil = '';  
    int count = 0;  
  
    for (int i = a.length - 1; i >= 0; i--) {  
        hasil = a[i] + hasil;  
        count++;  
        if (count % 3 == 0 && i != 0) {  
            hasil = '.' + hasil;  
        }  
    }  
    return hasil;  
}
```

Untuk bagian ini saya membuat cara sederhana untuk mengubah angka menjadi format ribuan. Pertama angka diubah menjadi String menggunakan `toString()`.

Setelah itu saya membaca angka dari belakang menggunakan perulangan dan menambahkan titik setiap tiga digit. Dengan cara ini, angka 15000 akan ditampilkan menjadi 15.000. Cara ini saya gunakan supaya harga yang tampil lebih mudah dibaca.



8. Tugas Aplikasi Daftar Kontak

Minimal 6 kontak yang terdiri dari nama, nomor telepon, dan email. Data disimpan dalam list yang berisi objek dari sebuah class.

Halaman utama menampilkan daftar kontak menggunakan ListView.builder dan ListTile. Avatar berisi huruf pertama dari nama kontak.

Huruf pertama nama ditampilkan di dalam CircleAvatar.

Saat salah satu kontak ditekan, aplikasi membuka halaman detail yang menampilkan data kontak dan menyediakan tombol untuk kembali.

9. Implementasi Tugas

```
class Kontak {  
    final String nama;
```

```

    final String telepon;
    final String email;
    const Kontak(this.nama, this.telepon, this.email);
  }

```

Data kontak saya simpan di dalam daftarKontak. Setiap data dibuat sebagai objek dari class Kontak dan memiliki tiga bagian, yaitu nama, nomor telepon, dan email.

```

return ListTile(
  leading: CircleAvatar(
    child: Text(kontak.nama[0]),
  ),
  title: Text(kontak.nama),
  subtitle: Text(kontak.telepon),
  trailing: const Icon(Icons.chevron_right),

```

Di halaman KontakPage, ListView.builder digunakan untuk menampilkan semua data kontak. Bagian leading berisi CircleAvatar dengan huruf pertama nama, kemudian nama kontak ditampilkan sebagai title dan nomor telepon sebagai subtitle. Di bagian kanan terdapat ikon panah sebagai tanda bahwa item tersebut bisa dipilih.

```

onTap: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => DetailKontakPage(kontak: kontak),
    ),
  );
},

```

Ketika kontak ditekan, bagian onTap menjalankan Navigator.push(). Objek kontak yang dipilih kemudian dikirim ke DetailKontakPage supaya data yang ditampilkan sesuai dengan kontak yang dipilih.

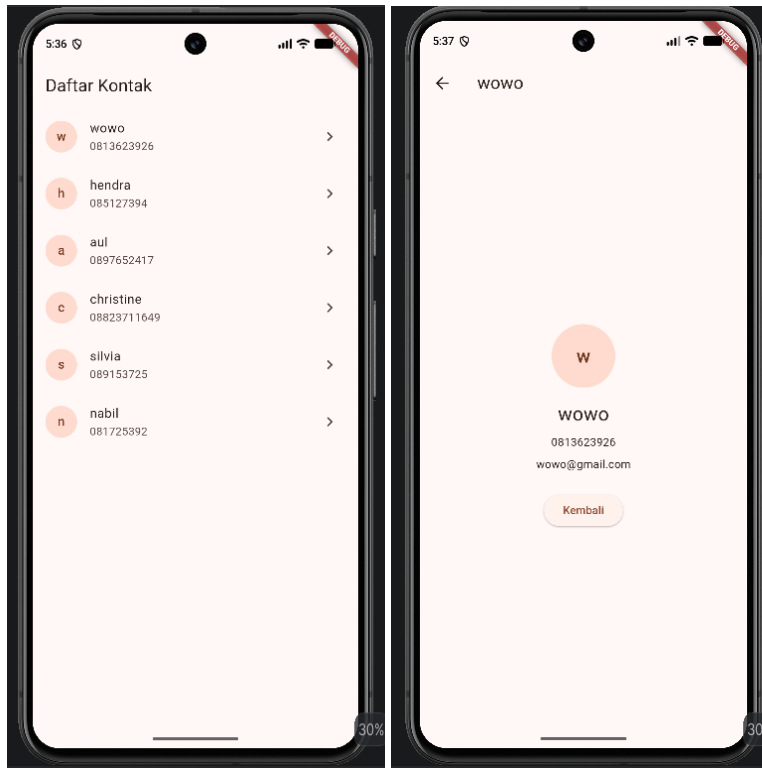
```

CircleAvatar(
  radius: 40,
  child: Text(kontak.nama[0], style: const TextStyle(fontSize: 24)),
),
const SizedBox(height: 16),
Text(kontak.nama, style: const TextStyle(fontSize: 24)),
const SizedBox(height: 8),

```

```
Text(kontak.telepon),  
const SizedBox(height: 8),  
Text(kontak.email),
```

Di `DetailKontakPage` saya menampilkan avatar dengan ukuran lebih besar, kemudian nama, nomor telepon, dan email. Dengan begitu, data kontak yang dipilih dapat dilihat lebih lengkap.



10. Kesimpulan

Setelah menyelesaikan praktikum ini, saya memahami bahwa beberapa widget Flutter dapat digabungkan untuk membuat tampilan aplikasi yang sederhana tetapi tetap terstruktur. Saya juga mencoba membuat class `Makanan` sebagai model data, lalu menampilkan datanya menggunakan `ListView.builder`, `Card`, dan `ListTile`.

Selain membuat daftar menu, saya juga mempraktikkan navigasi antarhalaman. `Navigator.push()` digunakan ketika saya memilih salah satu menu untuk membuka halaman detail, sedangkan `Navigator.pop()` digunakan untuk kembali ke halaman sebelumnya. Dari latihan ini saya jadi lebih memahami hubungan antara data, widget, dan navigasi dalam aplikasi Flutter.

11. Pertanyaan Refleksi

- Apa perbedaan `ListView` biasa dengan `ListView.builder`?
`ListView` biasa membuat item yang ditulis di dalamnya secara langsung, sehingga lebih cocok untuk jumlah item yang sedikit. Sedangkan `ListView.builder` membuat

item sesuai kebutuhan saat daftar ditampilkan atau di-scroll. Karena itu, ListView.builder lebih cocok digunakan untuk data yang jumlahnya banyak.

- Mengapa Row yang berisi teks panjang dapat menyebabkan overflow, dan bagaimana Expanded membantu?
Row menyusun widget secara horizontal. Jika teks di dalam Row terlalu panjang dan ruang yang tersedia tidak cukup, teks bisa keluar dari batas layar dan menyebabkan overflow. Expanded membantu dengan memberikan bagian sisa ruang kepada widget tersebut, sehingga teks dapat menyesuaikan lebar yang tersedia.
- Bagaimana data dikirim dari halaman daftar ke halaman detail pada praktikum ini?
Data dikirim melalui parameter pada constructor halaman detail. Saat item di daftar ditekan, Navigator.push() dipanggil dan DetailPage dibuat sambil mengirim objek Makanan yang dipilih. Dengan begitu, halaman detail bisa langsung menggunakan data dari item yang dipilih.